

分 类 号: TP391

单位代码: 10422

密 级: 公开

学 号: 200913029



山东大学

硕士学位论文

论文题目: 基于压缩感知的视频超分辨率重建及并行化

A GPU ACCELERATED ALGORITHM FOR COMPRESSIVE
SENSING BASED VIDEO SUPER-RESOLUTION

作 者 姓 名

吴西飞

学 院 名 称

计算机科学与技术学院

专 业 名 称

计算机软件与理论

指 导 教 师

向 辉 教 授

合 作 导 师

2012 年 4 月 20 日

原创性声明和关于论文使用授权的说明

原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的科研成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本声明的法律责任由本人承担。

论文作者签名：_____ 日 期：_____

关于学位论文使用授权的声明

本人完全了解山东大学有关保留、使用学位论文的规定，同意学校保留或向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅；本人授权山东大学可以将本学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或其他复制手段保存论文和汇编本学位论文。

论文作者签名：_____ 导师签名：_____ 日 期：_____

目 录

摘 要.....	I
Abstract.....	III
第一章、绪论.....	1
1.1 课题背景和研究意义.....	1
1.2 国内外研究现状.....	1
1.3 本文的思路及创新点.....	4
1.4 本文的组织结构.....	5
第二章、压缩感知理论简介.....	7
2.1 压缩感知理论提出的背景	7
2.2 压缩感知理论的数学模型.....	9
2.3 稀疏信号求解算法.....	11
2.4 压缩感知理论的应用	11
第三章、压缩感知理论在视频重建中的应用.....	13
3.1 程序流程.....	13
3.2 重建效果对比	14
3.3 程序的具体设计与实现.....	15
3.3.1 RGB 与 YCrCb 之间的转换.....	15
3.3.2 双三次插值的实现	15
3.3.3 Feather Sign 算法思想.....	17
3.3.4 三个阶段耗时及复杂度分析	18
3.4 并行计算的必要性和可行性.....	18
3.5 算法实现在各个阶段面临的问题及改进思路	19
第四章、GPU 的硬件特性及并程序优化	20
4.1 并程序编写的特点.....	20
4.2 GPU 的存储器层次及并程序的存储优化	20
4.2.1 存储器层次	21
4.2.2 存储器优化思路	22
4.2.3 共享存储器的优化	23
4.2.4 kernel 中变量数目的优化.....	25
4.3 减少数据在主机端和设备端之间的传递.....	26
4.4 配置优化与指令优化	28
4.5 各个阶段的加速比及分析.....	28
第五章、压缩视频编解码及其在视频超分辨率重建中的应用.....	30
5.1 视频编码格式与视频文件格式.....	30
5.2 MPEG-4 视频编解码思想	31
5.2.1 宏块的概念	31
5.2.2 帧间冗余的消除	32
5.2.3 帧内冗余的消除	33

山东大学硕士学位论文

5.2.4 变换编码与量化、熵编码.....	34
5.3 寻找帧间冗余信息.....	34
第六章、总结与展望.....	37
6.1 重建效果对比.....	37
6.2 本文工作总结.....	40
6.3 未来工作展望	41
参考文献.....	42
致谢.....	46
攻读学位期间发表的学术论文目录.....	47
攻读学位期间参加的项目.....	48

TABLE OF CONTENTS

Abstract In Chinese	I
Abstract In English	III
Chapter 1 Introduction	1
1.1 Background	1
1.2 Related Work	1
1.3 Research Contents Of Thesis.....	4
1.4 Structure Of Thesis	5
Chapter2 Theory Of Compressive Sensing.....	7
2.1 Background Of Compressive Sensing.....	7
2.2 Math Model Of Compressive Sensing	9
2.3 Algorithms For Solving Sparse Signals.....	11
2.4 Application Of Compressive Sensing.....	11
Chapter3 Video Super-resolution Based On Compressive Sensing.....	13
3.1 Flow Of Algorithm	13
3.2 Result Comparison	14
3.3 The Design And Implementation Of The Algorithm.....	15
3.3.1 Transform Between RGB And YCrCb.....	15
3.3.2 Bicubic Interpolation	15
3.3.3 Feather Sign	17
3.3.4 Analysis Of Three Stages	18
3.4 Necessity And Feasibility Of Parallel Computing	18
3.5 Problems And Solving Methods For Each Stage	19
Chapter4 Parallel Computing.....	20
4.1 The Feather Of Parallel Algorithm	20
4.2 Architecture Of Memory And Memory Optimization.....	20
4.2.1 Architecture of Memory.....	21
4.2.2 Memory Optimization.....	22
4.2.3 The Use Of Shared Memory	23
4.2.4 Optimization Of Kernel	25
4.3 Reduce Data Transform Between Host And Device	26
4.4 Other Optimization Methods.....	28
4.5 Analysis Of The Most Important Stages.....	28
Chapter5 Video Encoding And Frame Redundancy	30
5.1 Format Of Video.....	30
5.2 Main Idea Of MPEG-4	31
5.2.1 Concept Of MacroBlock.....	31
5.2.2 Interframe Redundancy Elimination.....	32
5.2.3 Intraframe Redundancy Elimination.....	33

山东大学硕士学位论文

5.2.4 Transform Coding, Quantization And Entropy Coding	34
5.3 Frame Redundancy	34
Chapter6 Conclusion And Future Work.....	37
6.1 Result Comparision	37
6.2 Conclusion.....	40
6.3 Future Work.....	41
References.....	42
Acknowledgements.....	46
The Thesis Published For The Master's Degree.....	47
The Projects Participated For The Master's Degree.....	48

摘 要

压缩感知理论是最近几年提出的基于信号稀疏性的理论,它为图形学领域的很多方向提供了新的思路。使用基于压缩感知理论重建的图像相对于传统算法重建的图像能保持更多的细节信息,获得更好的重建结果,本文采用的重建思想就是基于压缩感知理论的。

压缩感知理论本身的特点决定了重建过程是相对复杂的,以本文采用的视频为例,将分辨率为 256×256 的低分辨率视频重建为 512×512 的高分辨率视频,需要 3~4 秒的时间,但视频的实时性又要求每一帧的重建在很短的时间内完成(一般为几十毫秒),为了达到该目标,本文采用了如下优化策略:1)通过并行计算减少重建时间;2)减少数据在主机端和设备端之间的传递;3)通过研究视频解码,找到帧间冗余,减少重复计算。

显卡提供了能进行并行计算的 GPU,计算能力相对于 CPU 有了几十甚至上百倍的提高。并行计算的思想是将计算量大的部分通过并行计算的方式在 GPU 上实现,剩余的計算和少量的逻辑控制在 CPU 上实现,结果表明该模式相对于只使用 CPU 进行计算的模式加速了几十倍,为将重建算法应用到视频的实时重建提供了可能。

由于最终生成的结果是在 GPU 的全局存储器中,可以借助 OpenGL 和 CUDA 的交互,将生成的结果直接在显卡上显示出来,减少了数据在内存和显存之间的传递,提高了帧率。

日常常见的视频格式都是压缩视频,帧间冗余信息在视频的编解码过程中会有所体现。相邻的帧之间存在很多没有任何变化的块,对于这样的块可以保留上一帧的结果,而在下一帧中不需要进行重复计算,从而减少了计算量,进一步提高了重建帧率。

经过以上三种优化策略,实现了对分辨率为 256×256 的低分辨率视频的实时重建(平均 15 帧每秒),重建结果相对于常见播放器拉伸后的结果有了明显的改进。

关键词：图像超分辨率重建；视频超分辨率重建；压缩感知；GPU 加速；并程序优化；视频编解码。

ABSTRACT

Compressive sensing, also known as compressed sensing, is a technique motivated by recent results in sparse signal representation, which provides a new way to solve the problems in various field of traditional image processing. Compared to the traditional algorithm for image reconstruction, the algorithm based on compressive sensing can achieve a better result, a super-resolution image with more details, so the method of video super-resolution used in this paper is based on compressive sensing.

The algorithm of image reconstruction based on compressive sensing is complex. Take the image used in this paper for example, the process of reconstructing a high-resolution image with the size of 512×512 from the low-resolution image with the size of 256×256 takes 3~4 seconds, meanwhile, the time used for reconstruct a frame of video is restricted at a very short time (less than 100 milliseconds). In order to achieve this goal, three optimization strategies are used in this paper: 1) reduce time used for reconstruction by parallel computing; 2) reduce data transfer between host and device; 3) reduce the amount of computation by finding redundant data between adjacent video frames.

With the aid of GPU, the compute capability of PC can achieve dozens or even hundreds of times compared to the PC equipped with CPU. In the process of designing parallel program, the most time-consuming parts are completed in GPU in the form of parallel computing, others are implemented in CPU. The experiment of this paper shows that parallel program can achieve a speed up of thirty-five times than serial program, which makes it possible to implement the algorithm of super-resolution based on compressive sensing on real-time video reconstruction.

As the data of high-resolution image is allocated in global memory of GPU, the result can be shown directly on graphic card with the aid of interaction between OpenGL and CUDA, which reduce the data transfer between host and device, and

the frame-rate of video super-resolution is increased.

Most of the videos used in our daily life are compressed video, and the data redundancy between adjacent frames can find in the process of video decoding. There are many macro-blocks in p-frame with the mode of not_coded, which means that the data of the block in this frame is the same as the block of previous frame in the same position. As the reconstruction of previous macro-block has been completed, the computation of macro-block in this frame can be avoided. The amount of computation is reduced and the frame-rate of video super-resolution is increased.

After the three optimization strategies are implemented in the algorithm, we can achieve a reconstruction of low-resolution with the size of 256×256 at the frame-rate of fifteen fps, and the reconstructed result of video have a better performance compared to the commonly used media player(such as windows media player).

Key Words: Image super-resolution; compressive sensing; parallel computing; parallel algorithm optimization; video encoding; video decoding; video super-resolution.

第一章、绪论

1.1 课题背景和研究意义

在许多情况下，人们总是需要得到更高质量的图像。比如，高清晰度电视（HDTV）、视频监控、军用的目标识别和检测系统、民用的牌照阅读器、医学成像都需要高质量的图像。但是很多因素会导致图像质量的退化和细节部分的丢失，如采样过程中不可避免的会出现变形、模糊的现象；采样点不足导致只能获得低分辨率图像；图像放大过程中出现局部信息的不完善。

超分辨率重建就是为了解决这类问题而提出，它是指利用同一场景的一系列或一幅低分辨率图像来重建一幅高分辨率图像，利用低分辨率图像帧之间的相似性、冗余性及一些先验知识进行数据融合得到高分辨率图像。

超分辨率重建应用的场合非常广，提出的算法往往针对特定的场合。本文针对的是超分辨重建思想在“高清视频”上的应用。

在日常生活中，由于种种原因，我们只能获得低分辨率视频，比如，利用网络观看视频，由于网络的传输能力有限，在指定的时间内只能传送低分辨率的视频；由于技术原因，我们接受的电视信号只是标清信号；在录制视频时，由于录像设备的限制，只能得到一个低分辨率的视频等。

对于图像的放大，比较常用和简单的方法是进行插值，但是在插值过程中图像的细节部分并没有增加，高频部分的丢失导致图像变得模糊。针对这一现象，本文给出了基于压缩感知思想的重建算法，通过该算法得到的高分辨率图像比通过插值或其它重建算法得到的高分辨率图像能获得更多的细节信息。

1.2 国内外研究现状

视频可以分为压缩视频和未压缩视频，实际应用中最常见的视频都是压缩格式，如 MPEG 系列、H. 263、H. 264 等压缩标准，对于压缩视频的重建，目前主要有两种思路：第一种是针对压缩视频中引起图像质量退化的过程（如由于参数量化引起的图像质量下降、由于分块压缩引起的块效应等）应用重建算法，

减少每一帧的噪声；第二种是首先将视频进行解码，恢复成普通的序列影像后，再利用图像超分辨率重建的方法进行重建。

与图像重建相比，压缩视频与具有自己的特性：图像序列不再是像素的强度值，而是压缩的比特流；运动矢量提供了与时间相关的观测，使子像素的偏移产生噪声，这在传统的重建中是没有的；编码器的结构会带来块效应、振铃效应以及时域的闪烁等，这会直接影响到图像的主观效果。

Patti 和 Altunbasak[1]在 DCT 域内发展了一个针对 MPEG 压缩视频的 POCS 超分辨率重建方法，充分利用了压缩数据中的量化信息，之后，他们又进一步在空域发展了一个基于 MAP 框架的方法[2]，Segall 等人[3]系统的阐述了在空间域进行压缩视频超分辨率重建的原理与方法，并在 MAP 框架内实现运动估计和超分辨率影像的同时求解。Alvarez 等人[4]阐述了如何去除压缩视频超分辨率重建中不可观测像素的影响。

理论上，量化信息的使用必然会有效提高超分辨率重建的精度，但从目前的研究成果看，重建精度提高的程度并不是很大，而且考虑量化信息还会增加非常大的运算量。因此，本文的重建思想中没有考虑针对视频压缩过程中图像质量退化的重建，而是采用的传统的针对图像的超分辨率重建思想。图像超分辨率重建的发展现状如下。

图像超分辨率重建的基本思想最初是由 Harri[5]和 Goodma[6]于上世纪 60 年代在单幅图像复原的概念和方法的基础上逐步形成并发展起来的。Tsai 和 Huang[7]首次以傅里叶变换为基础框架，给出了一个利用多幅欠采样影像重建一幅高分辨率影像的频率域方法。

图像重建从解决问题的方式上总体归结为两大类：频率域方法和空间域方法。频率域方法是在频域内解决图像内插问题，其观测模型是基于 Fourier 变换的位移特性，最具有代表性方法是消混叠重建方法[8]。这类方法的缺点是只局限于全局平移运动和线性空间不变降质模型，忽略了模糊和噪声的影响，包含空域先验知识的能力有限。

空间域方法有更大的研究和应用价值，成为后来人们超分辨率技术研究的主流，空间域方法首先建立一个观测模型，认为低分辨率图像是高分辨率图像是经过变形、模糊、降采样等步骤得到的，解决的思想一般是通过各种方式来

获得这个观测模型的估计值, 利用估计的观测模型进行重建, 比较有代表性的方法有迭代反投影方法、凸集投影方法 (POCS)、最大后验概率方法 (MAP)、基于机器学习的方法以及在以上各种方法基础上进行改进和综合的方法。

Irani 和 Peleg[9]提出了迭代反投影方法, 在这个方法中, 首先用相对观测低分辨率图像的一个初始估计作为当前的高分辨率图像, 并把这个当前结果进行反投影获得低分辨率仿真图像, 低分辨率仿真图像与实际观测低分辨率图像的差值称为仿真误差, 根据仿真误差不断迭代更新当前估计, 最终使误差达到最小。

凸集投影方法 (POCS) 利用先验知识定义凸约束集合, 使高分辨率图像的解空间收敛于该凸约束中。凸约束集合有优良的特性, 如正定、能量有界、数据一致性、平滑等, 这些优良的特性正是高分辨率图像所期望的。Tekalp、Patti 等人[10]对 POCS 方法进行了深入的研究, 对凸集投影方法进行改进, 该算法考虑了以下几种因素: 照相机运动、非零孔径时间、每个传感器单元的非零物理尺寸、由成像光学元件引起的模糊、传感器噪声、任意空间时间采样网格等, 使重建效果得到明显改善。

最大后验概率 (MAP) 方法是基于概率统计理论的超分辨率图像重建方法, 是在已知低分辨率序列的前提下, 使出现高分辨率图像的后验概率达到最大。Borman 等人[11]提出一种能够同时进行多帧超分辨率视频重建的 MAP 估计方法。这种方法通过引入空间时间平滑约束和运动估计器可信度参数来提高超分辨率重建质量, 其特点是对运动估计误差具有较强的鲁棒性。

压缩感知理论的提出为图像超分辨率重建提供了一个新的思路。压缩感知理论是由 David Donoho、Emmanuel Candès 和陶哲轩等人提出, 是一个比较新的理论。Yi Ma[12][13]等人将该理论用到了图像的超分辨率重建上, 并将基于压缩感知的图像重建算法与传统的图像超分辨率重建算法进行了比较, 得出结论: 基于压缩感知的图像超分辨率重建可以得到比传统重建方法更好的效果。

NVIDIA 的显卡 (也可称为 GPU) 可以完美的支持标量运算, 并为开发者提供了编写并程序的类 C 语言 CUDA。2009 年, ATI 也提供了编写自己平台并行程序的类 C 语言 openCL。丰富的开发平台使得尝试编写在 GPU 上进行的并行程序变的容易, 由于压缩感知的信号重建过程相对复杂, 很多学者尝试通过并行

计算的方式减少重建所需时间。

M. Andreucut[14]在信号重建算法中, 尝试使用匹配追踪算法 (Matching Pursuit) 求解稀疏信号, 并实现了在 GPU 上运行的并程序, 结果显示: 在使用单精度类型变量运算的条件下获得了 31 倍的加速比, 使用双精度变量运算的情况下获得了 21 倍的加速比。

Alexandre Borghi[15]使用了 Moreau-Yosida 算法来解决一个简单的压缩感知问题, 充分利用了平行计算平台, 如共享存储器、矢量计算、多核, 将并程序与在多核 CPU 上运行的串程序进行对比, 结果证明: 在 GPU 上运行的程序获得了很高的加速比。

1.3 本文的思路及创新点

压缩感知理论是近几年提出的一个比较新的研究领域, 将压缩感知理论用于图像超分辨率重建可以获得比传统方法更好的效果, 但是压缩感知的信号重建过程相对复杂, 重建比较耗时, 以本文为例, 将 256×256 的低分辨率视频重建为 512×512 的高分辨率图像, 耗时 3~4 秒, 而视频的实时性要求每一帧的重建必须在较短的时间 (100 毫秒以内) 完成。

为了完成将压缩感知思想应用到视频的实时重建上, 本文主要采取了以下三种优化策略:

1) GPU 硬件设计非常适合并行, 其计算能力可以达到 CPU 的几百倍。通过对程序各个阶段的耗时进行分析, 将计算量大的部分通过并行计算的方式在 GPU 上实现, 剩余的少量计算和逻辑控制通过 CPU 实现, 该架构相对于仅使用 CPU 进行计算的方式, 重建速度达到了原来的 35 倍, 为将重建思想应用到视频帧的重建提供了可能。

2) 数据在主机端 (CPU 和内存) 和设备端 (GPU 和显存) 之传递是通过 PCI 接口实现的, 其传输带宽远小于内存和显存的带宽, 往往成为程序执行效率的瓶颈。程序的最终结果保存在 GPU 的全局存储器中, 借助 OpenGL 和 CUDA 之间的交互, 可以将生成结果直接通过显卡显示出来, 减少了数据在主机端和设备端之间的传递。相对于借助第三方库 (如 openCV) 进行显示的方式, 每一帧可以节省 20~30 毫秒的时间。

3) 压缩视频存在帧间冗余, 并且在视频的解码过程中可以得到冗余数据, 即当前帧相对于前一帧没有任何变化的部分, 对与该部分, 只需保留前一帧的数据, 而当前帧的相应部分无需进行重建。Xvid 是一种开源的 MPEG-4 格式的视频编解码器, 源码可以在其官方网站 (<http://www.xvid.org/>) 下载, 本文通过 Xvid 解码视频并得到帧间冗余的数据, 减少了计算量。

通过以上三种优化方案, 可以达到对 256×256 低分辨率视频实时重建, 重建视频比通过常见播放器 (如 Windows Media Player) 拉伸后的视频有更好的效果, 细节部分更加完善 (见第六章的效果对比)。

1.4 本文的组织结构

本文主要包括三大部分的内容: 压缩感知理论及其在超分辨率重建中的应用、GPU 编程及并程序优化、视频编解码及帧间冗余的寻找。

第 1 章为绪论, 主要介绍了视频超分辨率重建的课题背景及从事该方面研究的意义, 然后分别对超分辨率重建的国内外研究情况进行了描述, 接下来介绍了本文的思路及创新点, 最后是论文的组织结构。

第 2 章主要介绍了压缩感知理论。首先介绍了压缩感知理论提出的背景, 然后说明了压缩感知的数学模型, 接着介绍了模型的核心部分: 稀疏信号的求解, 并对常见的求解算法进行了分析, 最后列举了压缩感知理论在图像处理的各个研究方向的应用。

第 3 章具体介绍了压缩感知理论在视频重建中的应用。首先介绍了一下程序的流程, 然后将基于压缩感知思想的重建算法与传统重建算法进行比较, 接着对算法进行了详细的分析, 并分析了并程序设计的可行性和必要性, 最后总结了算法实现在各个阶段面临的问题及改进思路。

第 4 章介绍了 GPU 的硬件特性及并程序优化。首先总结了并程序编写相对于串程序编写的特点, 接着分析了 GPU 的存储器层次及并程序的存储优化策略, 然后介绍了减少数据在主机端和设备端之间的传递的几类方法, 最后列出了各个阶段的加速比, 并对加速结果进行了分析。

第 5 章对视频的编解码进行了分析。首先简介了视频编码格式和视频文件格式的区别, 然后对 MPEG-4 编解码思想进行了分析, 最后介绍了如何找到压缩

视频帧之间的冗余，从而达到减少计算量的目的。

第 6 章，对本文内容进行总结。首先将重建后的图像和播放器解码后的图像效果进行了对比，接着对本文的算法进行了分析和总结，最后给出了算法可以进一步优化和改进的地方。

第二章、压缩感知理论简介

2.1 压缩感知理论提出的背景

压缩感知 (CS, Compressive Sensing) 理论是在已知信号具有稀疏性或可压缩性的条件下, 对信号数据进行采集、编解码的新理论, 该理论表明, 当信号具有稀疏性或可压缩性时, 通过采集少量的信号投影值就可实现信号的准确或近似重构。该理论的提出克服了传统编解的种种缺陷[20]。传统编解码理论框图如下:

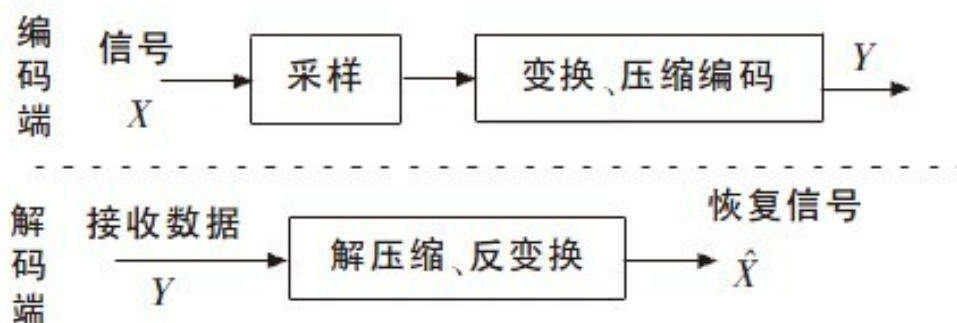


图 2-1 传统编解码框架

编码端先对信号进行采样, 再对所有采样值进行变换, 并将其中重要系数的幅度和位置进行编码, 最后将编码值进行存储或传输; 信号的解码过程仅仅是编码的逆过程, 接收的信号经解压缩、反变换后得到恢复信号。这种传统的编解码方法存在三个缺陷:

1) 传统的 Nyquist 采样定理要求信号的采样率不得低于信号带宽的两倍, 这给采样设备的信号处理能力提出了更高的要求, 使得硬件系统面临着很大的采样速率的压力。

2) 在压缩编码过程中, 大量变换计算得到的小系数被丢弃, 造成了数据计算和内存资源的浪费。

3) 经典的数据压缩技术, 都是从数据本身的特性出发, 寻找并剔除数据中隐含的冗余度, 从而达到压缩的目的, 压缩的过程需要复杂的算法来完成。相较而言, 解码过程反而一般来说在计算上比较简单, 以音频压缩为例, 压制一

一个 mp3 文件的计算量远大于播放（即解压缩）一个 mp3 文件的计算量。这种压缩和解压缩的不对称性正好同人们的需求是相反的。在大多数情况下，采集并处理数据的设备，往往是廉价、省电、计算能力较低的便携设备，例如傻瓜相机、或者录音笔、或者遥控监视器等等。而负责处理（即解压缩）信息的过程却反而往往在大型计算机上进行，它有更高的计算能力，也常常没有便携和省电的要求。也就是说，我们是在用廉价节能的设备来处理复杂的计算任务，而用大型高效的设备处理相对简单的计算任务。

压缩感知理论的信号编解码框架和传统的框架大不一样，如下图所示。

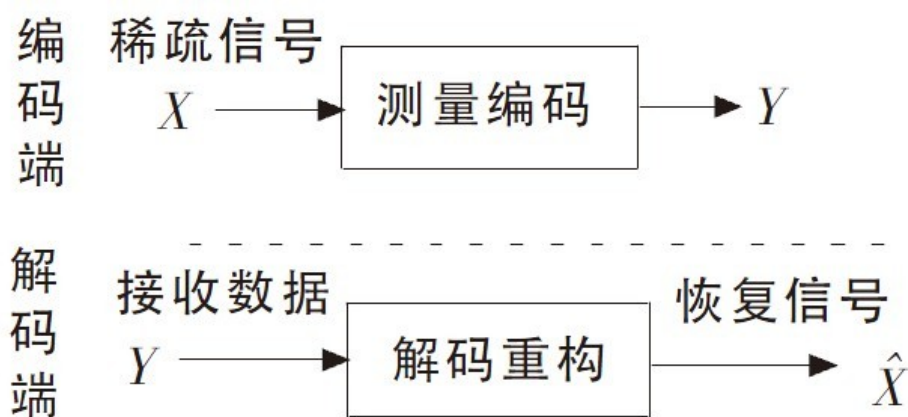


图 2-2 压缩感知理论的编解码框架

该理论对信号的采样、压缩编码发生在同一个步骤，利用信号的稀疏性，以远低于 Nyquist 采样率的速率对信号进行非自适应的测量编码。测量值并非信号本身，而是从高维到低维的投影值，解码过程不是编码的简单逆过程，而是在盲源分离中的求逆思想下，利用信号稀疏分解中已有的重构方法在概率意义上实现信号的精确重构或者一定误差下的近似重构[19]，解码所需测量值的数目远小于传统理论下的样本数。

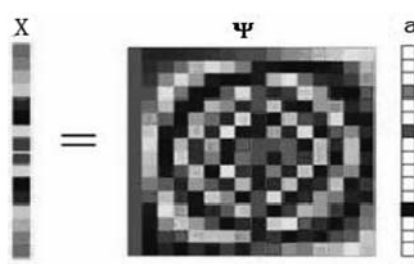
压缩感知的编解码理论克服了传统编解码理论的缺陷：1) 信号的采集可以在远低于 Nyquist 采样率的速率下进行；2) 采集的是原始信号在低维上的投影信号，节约了所需的设备内存；3) 信号的采集压缩过程结合在一起，采集的即使压缩后的数据，压缩所需的计算量大大降低。

2.2 压缩感知理论的数学模型

压缩感知理论是由 Candès [16], Romberg [17], Tao [18], and Donoho [19] 等人提出。它指出, 只要信号是可压缩的或在某个变换域是稀疏的, 那么就可以用一个与变换基不相关的观测矩阵将变换所得高维信号投影到一个低维空间上, 然后通过求解一个优化问题就可以从这些少量的投影中以高概率重建出原信号, 可以证明这样的投影包含了重建信号的足够信息。

信号的稀疏性是压缩感知理论的基础之一, 对于一个信号, 如果只有少数元素是非零的, 则该信号是稀疏的。通常时域内的自然信号都是非稀疏的, 但在某些变换域可能是稀疏的。例如, 对于一幅自然图像, 几乎所有的像素值都是非零的, 但是将其变换到小波域时, 大多数小波系数的绝对值都接近于零, 并且有限的大系数能够表示出原始图像的绝大部分信息。

非稀疏信号变换到对应的稀疏域可以通过稀疏矩阵来实现, 稀疏矩阵定义如下: 如果一个信号 X 可以表示为矩阵 Ψ 列向量的线性组合, a 是系数序列, 如果 a 只有很少的大系数, 则称 X 是可压缩的 (稀疏的), 如果 a 中只有 K 个元素非零, a 为 X 的 K 稀疏表示, 稀疏矩阵设计使 K 值尽量的小, 下图中 a 就是 X 的 3 稀疏表示, Ψ 为 X 的稀疏矩阵。

$$X = \sum_{k=1}^N \Psi_k \alpha_k = \Psi \alpha$$


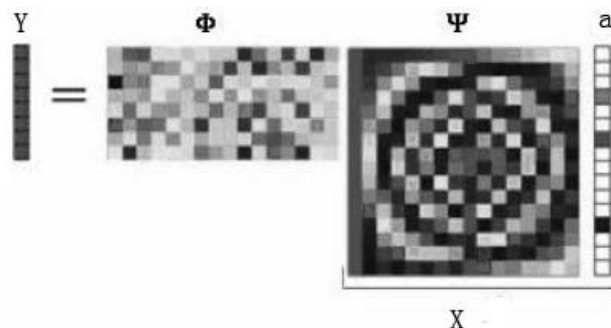
用基 Ψ 进行稀疏表示 (3稀疏)

图 2-3 信号的稀疏矩阵

如何设计稀疏矩阵从而得到原始信号的最稀疏表示, 是压缩感知理论面临的问题之一。对于图像信号, 常用的稀疏矩阵有正 (余) 弦基、小波基、chirplet 基以及 curvelet 基等, 另外通过机器学习的方式也可以获得一类信号的稀疏矩阵。不同信号类型的稀疏矩阵是不同的, 最优的稀疏矩阵需要得到信号的最稀疏表示。

图 2-4 中 Φ 为测量矩阵，它的作用是：原始信号通过测量矩阵得到它的投影信号（即压缩后的信号），在信号的采集过程中使用。

公式： $y = \Phi X$



压缩传感线性测量过程

图 2-4 压缩传感线性测量过程

其中，假设原始信号 X 的元素个数为 N ，压缩信号 Y 的大小为 M ($M \ll N$)，则稀疏矩阵 Ψ 大小为 $N \times N$ ，测量矩阵 Φ 大小为 $M \times N$ 。

从图 2-4 中，就可以看到压缩感知理论编解码的原理。

信号压缩： X 与测量矩阵 Φ 相乘得到 Y 值。

信号重建：根据压缩后的信号 Y 重建原始信号 X 的过程：

(1) 根据 $y = Aa$ ($A = \Phi \Psi$ ，称为传感矩阵)，求出原始信号 X 的最稀疏表示 a 。

(2) $X = \Psi a$ ，得到原始信号。

根据压缩信号 Y 求原始信号 X ，是一个病态问题，求出的 X 有无穷多个解，但是求得的 X 的最稀疏表示是唯一的。理论基础是 Candes [16] 和 Donoho [19] 给出的证明：如果传感矩阵满足约束等距性，也就是是测量矩阵和稀疏矩阵不相关，那么可以从 y 精确重构出 a 。且 a 的最优解是唯一的。

压缩感知理论编解码原理可以发现，信号的编码过程是非常简单的，就是原始信号在传感矩阵上进行投影。但是信号重建的过程需要求解原始信号的最稀疏解，即在公式 (1) 中根据压缩信号 y 求解原始信号最稀疏解 a 的过程，这是信号重建过程最复杂的部分。

可见，稀疏信号求解算法是压缩感知理论的核心，下面对稀疏信号求解算法进行一下总结。

2.3 稀疏信号求解算法

稀疏信号求解的问题就是求解 $\mathbf{a}$ 最小 L_0 范数问题，范数的定义如下：

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^N |x_i|^p \right)^{1/p}$$

可见，求解最小 L_0 范数问题就是求解非零元素个数最少的 $\mathbf{a}$ 值。但 Donoho 指出，最小 L_0 范数问题是一个 NP-hard 问题，需要穷举 $\mathbf{x}$ 中非零值的所有 C_n^k 种排列可能（ n 为稀疏信号的长度， k 为非零元素的个数），因而无法求解，但是可以转化为求解 L_1 范数的问题[21][24]。最小 L_1 范数就是求解 $\mathbf{a}$ 所有元素和的最小值。常见的求解最小 L_1 范数的方法有，内点法[25]、梯度投影法[26]以及同伦算法[27]等。

匹配追踪算法(Matching Pursuit, MP)是一种贪婪迭代算法，其基本思想在每一次的迭代过程中，从完备原子库里（即传感矩阵 $\mathbf{A}$ ）选择与信号最匹配的原子来构建稀疏逼近，并求出信号表示残差，然后继续选择与信号残差最为匹配的原子，经过一定次数的迭代，信号可以由一些原子线性表示。但是由于信号在已选定原子(测量矩阵的列向量)集合上的投影的非正交性使得每次迭代的结果可能是次最优的，因此为获得收敛可能需要经过较多次迭代。正交匹配追踪算法[28](Orthogonal Matching Pursuit, OMP)则有效克服了这个问题，该算法沿用了匹配追踪算法中的原子选择准则，只是通过递归地对已选择原子集合进行正交化以保证迭代的最优性，从而减少了迭代次数。

此外还有最小角度衰减(LARS, Least Angle Regression)算法[29]、符号特征算法(Feather Sign)[30]等。由于 Feather Sign 算法耗时较短，所以本文选用 Feather Sign 算法进行稀疏信号求解。

2.4 压缩感知理论的应用

压缩感知理论的提出为很多图像处理问题给出了新的解决方案[36]。

基于压缩感知的信号采集编码相对与传统的信号采集方式，减少了信息的采集量和压缩所需的计算量，可以在较短的时间完成大量信息的采集。在医学上，磁共振成像检测过程中，使用压缩感知技术进行数据采集，可以大大缩短

采集信息所需的时间，减少了患者身体的伤害。在军事上，“数字化战场”的实现要求捕捉战场的全部信息，传统的采样方式成本比较昂贵，使用“压缩感知”技术减少采样设备的采样点的数目，这样可以降低采样的成本，为获得战场的全部信息提供了可能。

压缩感知采样对昂贵的成像器件的设计产生重大影响，对目标信号将有望采用少量的随机观测次数就能获得高精度重构。比如地震勘探成像和核磁共振成像中使用的成像设备。美国 Rice 大学已经研制出基于该理论的“单像素相机”。

从压缩传感理论可知，少数的测量值能够保持原始信号的结构和足够多信息。因此，少量的测量值可以直接用于实现各种图像处理任务，如图像分类、特征提取、目标检测以及信息融合等，并且由于测量值数目较少，信息密度高，可以大大减少相关算法的时间和存储代价。

马毅等人[22]将信号在稀疏矩阵上的稀疏表示，用于人脸识别（Robust Face Recognition via Sparse Representation），他们已经证明，一个随机提取的特征点集（一个包含了几百个点的稀疏表示）可以提供足量的信息来确定一幅图像在一个图像集中所对应的对象，而且这种方法能够识别损毁和遮挡的人脸，具有足够鲁棒。杨建超等[23]将稀疏表示应用到图像分类中，使用基于局部特征描述子的稀疏表示描述图像特征，减少的算法的复杂度，并获得了较好的效果。

Yi Ma[12][13]等人将该理论用到了图像的超分辨率重建上，并将基于压缩感知的图像重建效果和传统的图像超分辨率重建方法，如迭代反投影，基于机器学习的方法等，进行了比较，得出结论：基于压缩感知的图像超分辨率重建可以得到比传统重建方法更好的效果。本文就是基于该思想进行改进的。

第三章、压缩感知理论在视频重建中的应用

本文的工作思路是首先将压缩视频解码，对解码后的视频帧进行基于压缩感知的超分辨率重建得到分辨率更高的图像。

3.1 程序流程

图像的超分辨率重建相当于压缩感知的逆过程,即由压缩信号求原始信号。将低分辨率图像视为压缩信号 Y ，高分辨率图像视为原始信号 X ，通过稀疏信号求解的方法进行重建。

参考 2.2 节中信号重建过程公式，公式中需要用到传感矩阵 A ，和稀疏矩阵 Ψ 。根据 $y=Aa$ ($A=\Phi\Psi$ ，称为传感矩阵)，求出原始信号 X 的最稀疏表示 a ，然后根据 $X=\Psi a$ 得到原始信号 X 。具体实现过程如下图：

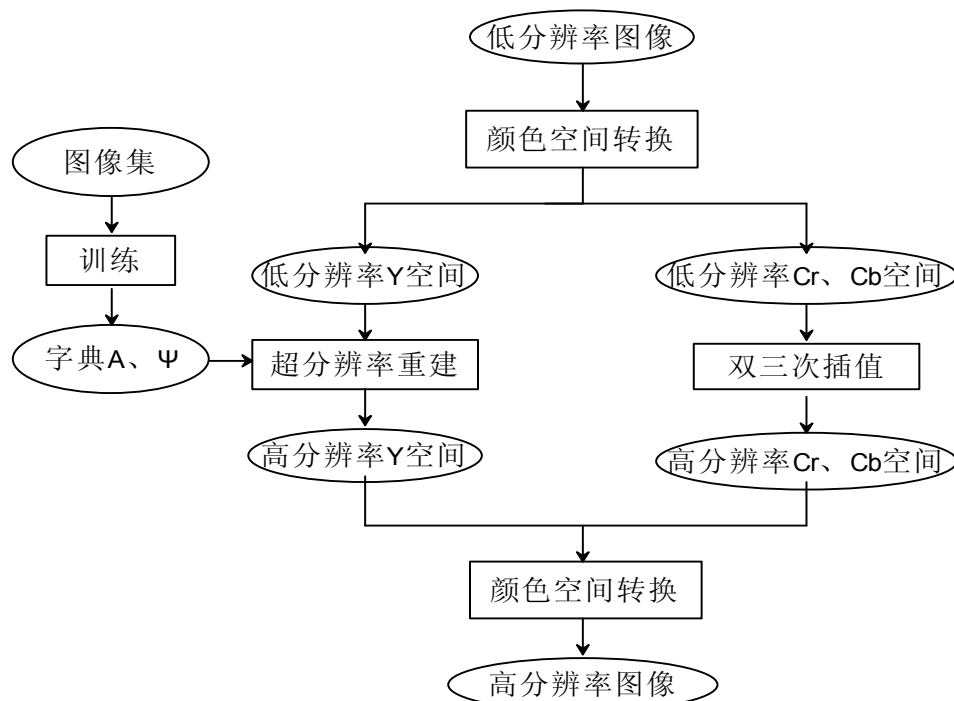


图 3-1 程序实现步骤

公式中用到的传感矩阵 A 和稀疏矩阵 Ψ 是通过机器学习的方式训练得到的。算法首先将低分辨率图像从 RGB 空间转换到 YCrCb 空间，因为在 YCrCb 空间中，图像的大部分信息保存在 Y 空间，只对低分辨率图像的 Y 空间进行超

分辨率重建，Cr、Cb 空间通过对低分辨率图像 Cr、Cb 空间进行双三次插值得到，从而减少计算量。

Y 空间的重建是通过将低分辨率 4×4 的块重建为高分辨率 8×8 的块：低分辨率中 4×4 的块视为压缩信号，对应的高分辨率 8×8 的块视为原始信号，通过 2.2 节中信号重建过程公式进行求解。可见，块是重建的基本单位，每个块重建的算法都是相同的，稀疏信号的求解是重建算法的关键。

3.2 重建效果对比

基于压缩感知思想的超分辨率重建可以获得比传统重建算法更好的效果。下图是基于压缩感知的图像重建算法与双三次插值和基于机器学习重建算法效果的比较。

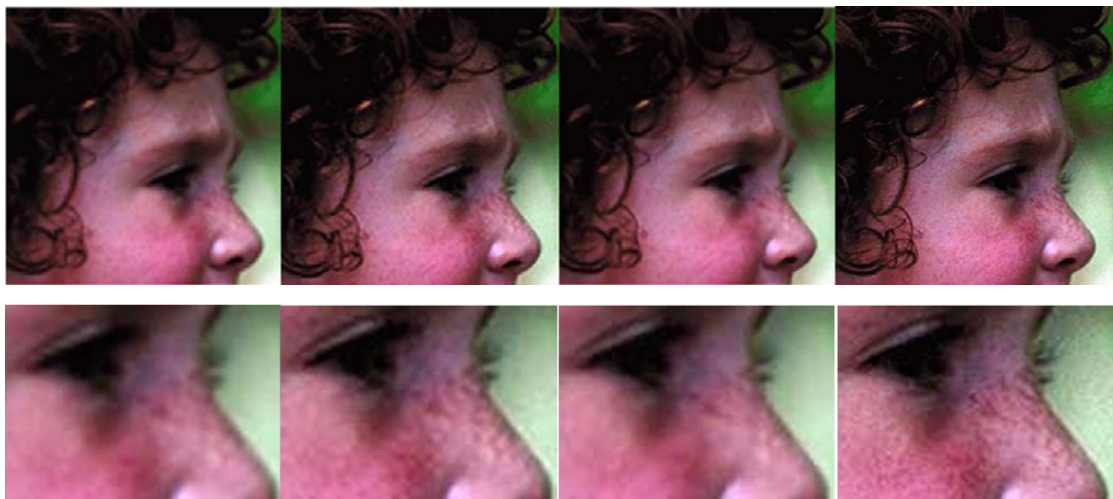


图 3-2 不同重建算法效果及细节效果对比（从左到右依次为双三次插值重建结果、基于压缩感知的重建结果、基于机器学习的重建结果、原始图像。上面一行为整体效果对比，下面一行为细节对比）

重建效果从两个方面进行评价：1) 基于人眼的主观识别，主要看两个方面：轮廓是否清晰、细节部分是否详细；2) PSNR 值， $PSNR = 10 \times \log_{10}(255^2 / MSE)$ ，MSE 是原图像与重建图像像素值之间均方误差，所以 PSNR 值越大，就代表失真越少。

从图 3-2 的效果对比可以发现，基于压缩感知理论重建后的图像中，女孩脸上的斑点更清晰，鼻子的轮廓也比较明显。证明重建后的细节部分更丰富，图像高频部分得到更好的恢复，下面从重建图像的 PSNR 值进行比较：

重建类型	PNSR（峰值信噪比）值
基于机器学习	32.820173 dB
双三次插值	33.872552 dB
基于压缩感知	34.640145 dB

可见，基于压缩感知理论重建的图像 PNSR 值更高。

通过两个方面比较可以得出结论：基于压缩感知理论的图像超分辨率重建能获得比传统重建方法更好的结果。

3.3 程序的具体设计与实现

从图 3-1 中可以看出，程序流程主要有三个部分：RGB 与 YCrCb 之间的转换、双三次差值、稀疏信号求解，下面分别对这三个过程进行分析。

3.3.1 RGB 与 YCrCb 之间的转换

1) 图像从 RGB 空间到 YCrCb 空间的转换公式如下：

$$\begin{aligned} Y &= 65.481 \times R \div 255 + 128.553 \times G \div 255 + 24.966 \times B \div 255 + 16 \\ Cr &= 112 \times R \div 255 - 93.786 \times G \div 255 - 18.214 \times B \div 255 + 128 \\ Cb &= -37.797 \times R \div 255 - 74.203 \times G \div 255 + 112 \times B \div 255 + 128 \end{aligned}$$

2) 图像从 YCrCb 空间到 RGB 空间的转换公式如下：

$$\begin{aligned} R &= Y \times 1.1644 + 1.5960 \times Cr - 222.9216 \\ G &= Y \times 1.1644 - 0.3918 \times Cb - 0.8130 \times Cr + 135.5754 \\ B &= Y \times 1.1644 + 2.0172 \times Cb - 276.8363 \end{aligned}$$

3.3.2 双三次插值的实现

插值是最常用的图像缩放算法。插值就是利用某种算法来估计由于图像放大而需要增加的像素点的值或由于图像缩小所需要去除的像素值。常见的插值算法有最近邻插值、双线性插值和双三次插值。其中，双三次插值得到的图像效果最平滑，但是计算量相对较大。

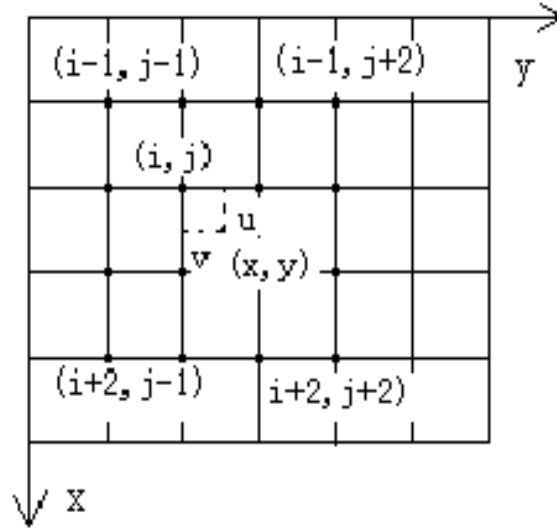


图 3-3 双三次插值图示

双三次插值用到了插值点周围的 4×4 邻域, 算法先分别对水平方向的四个点进行插值, 得到四个数据, 然后再对这四个数据进行插值, 插值公式如下。

```
//水平插值
float cubicInterpolate(float p[4], float v)
{
return p[1] + 0.5 * v*(p[2] - p[0] + v*(2.0*p[0] - 5.0*p[1] + 4.0*p[2] - p[3] +
v*(3.0*(p[1] - p[2]) + p[3] - p[0])));
}
//将水平插值后的结果进行竖直方向的插值
float bicubicInterpolate(float p[4][4], float u, float v)
{
float arr[4];
arr[0] = cubicInterpolate(p[0], v);
arr[1] = cubicInterpolate(p[1], v);
arr[2] = cubicInterpolate(p[2], v);
arr[3] = cubicInterpolate(p[3], v);
return cubicInterpolate(arr, u);
}
```

$p[4][4]$ 是插值点 (i, j) 周围 4×4 邻域, 共 16 个点, 如图 3-3, 该邻域是在 x 方向上的坐标从 $(i - 1)$ 到 $(i + 2)$, 在 y 方向的坐标为从 $(j - 1)$ 到 $(j + 2)$ 的 4×4 大小的块。

图中 i, j, u, v 的值通过下面的计算过程获得:

```
//获得缩放比
float scl_inv_x = (float)(src_h - 1) / (dst_h - 1);
float scl_inv_y = (float)(src_w - 1) / (dst_w - 1);
i = dst_i * scl_inv_x;
```

```

jj = dst_j * scl_inv_y;
//整数部分为坐标值
i = ((int)ii);
j = ((int)jj);
//小数部分为参数 (u, v)
u = ii - i;
v = jj - j;

```

上式中, $\text{src_h}(\text{src_w})$ 为原图像的高度 (宽度), $\text{dst_h}(\text{dst_w})$ 为插值后图像的高度 (宽度), scl_inv_x (scl_inv_y) 为在垂直 (水平) 方向的缩放比。

(dst_i , dst_j) 为目标图像的坐标, 分别与 scl_inv_x 和 scl_inv_y 相乘后, 整数部分为 (i , j) 值, 小数部分为 (u , v) 值。

另外需要注意的是: 在图像缩小的过程中, 为了防止图像出现锯齿化, 需要先对原图像进行低通滤波, 一般选择的是 5×5 的高斯低通滤波器; 当 4×4 邻域的坐标超出边界时, 选择离该坐标最近的边界值, 即当坐标值小于 0 时, 设为 0; 坐标值大于边界值时, 设为边界值。

3.3.3 Feather Sign 算法思想

Feather Sign 求解信号最稀疏解的约束函数如下:

$$\min_x f(x) \equiv \|y - Ax\|^2 + \gamma \|x\|_1,$$

假设 $x = [x_1, x_2, \dots, x_n]^T$, 且 $x_1 \geq 0, x_2 \leq 0, x_n \geq 0$,

那么, 有

$$\min_x f(x) \equiv \|y - (A_1 x_1 + A_2 x_2 + \dots 0 \dots + A_n x_n)\|^2 + \gamma \|x_1 - x_2 + \dots 0 \dots + x_n\|$$

,

这是一个关于 x 的二次方程, 可以在闭合形式求解。

算法: 1) 重复的猜测 x_i 的符号 (+, -, 0, ...) ; 2) 在闭合形式求解 x_i , 并重新猜测其符号。

很明显, 约束函数必须同时满足两个条件: 1) 系数 x 尽可能的稀疏; 2) 目标函数值尽可能的小。

3.3.4 三个阶段耗时及复杂度分析

在 2.7GHz 双核 CPU、2G 内存的环境下，对分辨率为 256×256 的图像重建为 512×512 的高分辨率图像，各个阶段的耗时如下表：

阶段	耗时（单位：ms）
RGB 与 YCrCb 的转换	17
双三次插值放大图像	297
Y 值重建	2875
总体	3203

在三个过程中稀疏信号求解过程是最复杂、耗时最多的，RGB 与 YCrCb 的转换耗时最少。

3.4 并行计算的必要性和可行性

从 3.3.4 节的中知道，重建一帧需要 3203 毫秒的时间（另外，视频解码和显示也需要消耗一定的时间），相对于视频 10~30 帧每秒的要求有很大差距，有必要借助 GPU 较强的计算能力缩短重建时间。

适合并行的程序一般具有如下特点：能找到大量存在的相同过程；各个过程之间不存在数据依赖。

图像的超分辨率重建就特别适合并行程序的编写。为了方便信号的重建，以块为单位进行重建，本文使用的是从低分辨率 4×4 的块重建为高分辨率 8×8 的块，然后对每一个块进行基于压缩感知的超分辨率重建，共需要进行 128×128 次相同的重建过程（相邻块之间有重叠）。由于各个块重建的算法是相同的，并且各个块之间不存在数据的依赖，所以程序是很适合并行的。

RGB 与 YCrCb 转换和双三次插值过程耗时很少，但是结果表明，对该阶段重建仍然可以获得一定的加速比，并且可以减少数据在主机端和设备端之间的交换，所以选择对这三个阶段都进行重建。

3.5 算法实现在各个阶段面临的问题及改进思路

程序运行的环境是：主机端(Intel Core2 2.7GHz、2G 内存)、设备端(NVIDIA Geforce GTS 450)，针对的是从分辨率为 256×256 的低分辨率图像重建为分辨率为 512×512 的高分辨率图像，各个阶段的工作思路如下表：

改进思路	程序实现	重建效果
实现基于压缩感知的图像超分辨率重建及并行化	压缩感知最重要的部分是稀疏信号求解，使用的是最小角度衰减法 (LARS, Least Angel Regression)。	串行程序 (即只使用 CPU 进行计算) 大约需要 60s 的时间，并行后大约需要 10s，加速比为 6。
由于 LARS 算法比较耗时，寻找到新的稀疏信号求解方法，并实现并行算法。	使用符号特征算法 (Feather sign) 进行稀疏信号求解。	串行程序大约需要 3200ms，并行后大约 90ms，加速比 35。
将算法用到视频帧的重建上。	openCV 解码视频 + 并行重建 + openCV 和 MFC 显示视频。	显示一帧大约需要 125ms，约 8 帧/秒。
由于是将 GPU 上的重建结果传递到内存后，通过 openCV 显示，增加了数据在显存和内存之间传递的次数，考虑直接通过 OpenGL 将 GPU 上的数据显示出来。	openCV 解码视频 + 并行重建 + OpenGL 显示视频。	重建一帧减少了 20~30ms 的时间，达到 10 帧/秒。
由于视频相邻帧之间的相关性，相邻帧之间有数据冗余，通过研究视频的解码寻找到这种冗余，减少计算量。	通过分析 Xvid 源码，找到帧间的冗余，对于上一帧的冗余，下一帧不需要重建。Xvid 解码视频 + 并行重建 + OpenGL 显示。	重建时间因冗余量的多少而不同，比如有的帧有 50% 的数据和上一帧是相同的，需要 50ms 重建完成，可以达到 20 帧/秒。

接下来两章的主要内容如下：第四章主要介绍并行程序的编写与优化，并对各个阶段的加速效果进行了对比和分析；第五章介绍了压缩视频的编码思想，如何寻找帧间冗余，并利用帧间冗余减少视频重建过程中的计算量。

第四章、GPU 的硬件特性及并程序优化

由于散热及本身架构等问题，CPU 的计算能力发展比较缓慢，但是 GPU 的出现大大提高了计算机的计算性能，GPU 是通过并行计算的架构实现计算。CPU 的控制和少量串行计算加上 GPU 的并行计算，该架构使 PC 的计算能力得到的极大的提高。并行程序的编写和优化[31]与 GPU 的硬件架构和特性密切相关，所以编写并行程序之前有必要了解一下 GPU 的硬件特性。

4.1 并行程序编写的特点

并行程序的编写相对于串行程序的编写要复杂一些，主要体现在以下几个方面。

1) 编程的思路要改变，一个 kernel 可能会被 N 个线程同时执行，kernel 的编写要考虑每一个线程的执行情况，防止指针越界、多个线程重复操作、读写冲突等各种情况；

2) 并行程序的优化。并行程序的优化策略比较多，一个没有优化过和优化过的并行程序，时间上可能会相差几倍甚至几十倍。

3) 调试困难，并行程序写在一个单独的文件，由特定的编译器去编译，无法添加断点，一般的解决方法是将中间结果复制到内存输出，逐步分析数据的正确性。程序出现错误后的提示也是非常有限的；

4) 并行程序出现指针越界等问题的后果比串行程序要复杂。指针越界很容易导致显卡驱动的崩溃、机器自动重启；

4.2 GPU 的存储器层次及并行程序的存储优化

GPU 中存在多种类型的存储器，如全局存储器、共享存储器、寄存器等，都可以在并行程序中使用，并行程序的优化需要充分考虑不同类型存储器的特点。

4.2.1 存储器层次

CUDA 线程在执行过程中可以访问多个存储器空间的数据，如下图所示。每个线程都有一个私有的寄存器 (register)，每个线程块都有一个共享存储器 (shared memory)，该存储器对于块内的所有线程都是可见的，并且与块具有相同的生命周期。最终，所有线程都可访问相同的全局存储器。此外还有两个只读的存储器空间，可由所有线程访问，这两个空间是常量存储器空间 (Constant memory) 和纹理存储器空间 (Texture memory)。对于同一个应用程序启动的内核而言，全局、固定和纹理存储器空间都是持久的，寄存器在线程执行完后即释放，共享存储器在线程块中的所有线程执行完成后自动释放。

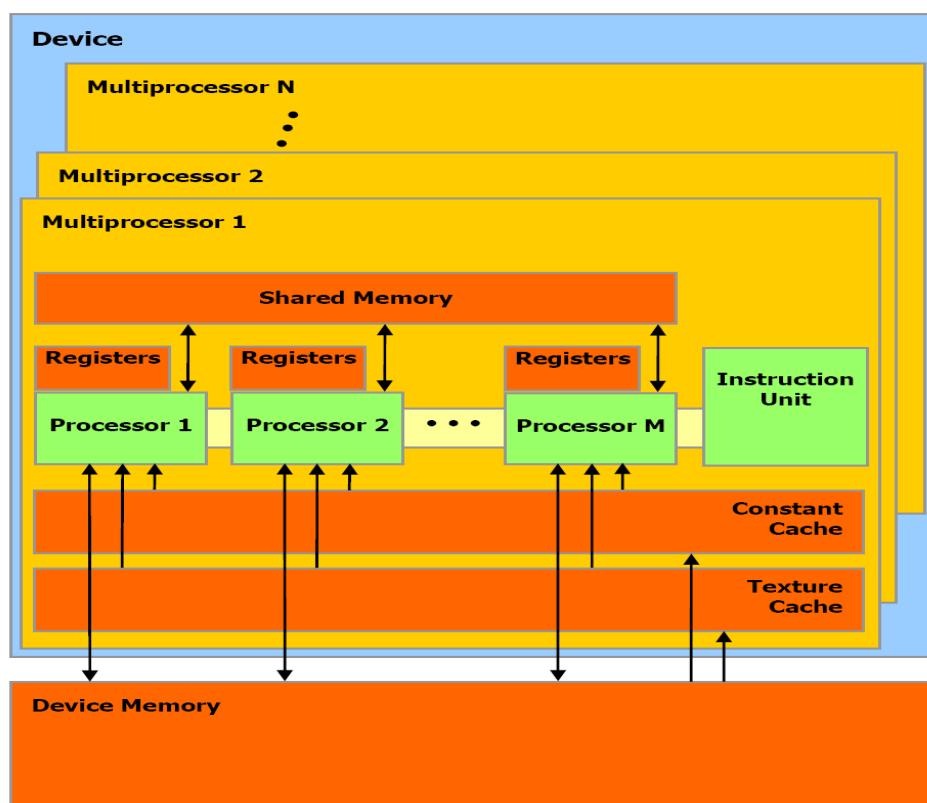


图 4-1 GPU 存储器层次

最靠近流处理器的是寄存器文件 (register file)，每个寄存器文件是 32bit，对线程来说，寄存器都是私有的，不允许其它线程访问。由于更靠近流处理器，寄存器具有最快的速度，CUDA 的运行环境能够动态的为线程块分配寄存器，而每个线程块中的线程占用的寄存器大小则是静态分配的，在线程块寿命期间都不会更改，因此一个线程占用的寄存器数目是它在运行时占用的最大

数目。如果寄存器被消耗完，数据将被存储在本地存储器（local memory）。

对每个线程来说，本地存储器也是私有的，但是本地存储器是显存中的一个分区，速度很慢，而且使用本地存储器过多的话，程序也会终止，因此，编程时要尽量保证不将数据放到本地存储器中，这可以通过修改块大小，使用共享存储器等方法来解决。

共享存储器是可以被同一块中的所有线程访问的可读写存储器，它的生存期就是块的生命期。在没有冲突的情况下，访问共享存储器几乎与访问寄存器一样快，是实现线程间通信的最好方法。共享存储器可以实现许多不同的功能，如用于保存共用的计数器或者块内的公用结果（例如 reduction）。在同一个块内，所有的线程都能够读写共享存储器中的数据。一般而言，在 kernel 运行时，要先将数据从全局存储器写入共享存储器；计算完成后要将共享存储器中的结果转存入全局存储器。

常数存储器（constant memory）和纹理存储器（texture memory），它们是利用 GPU 用于图形计算的专用单元实现的。常数存储器空间较小（只有 64KB），属于片外存储器，其速度比共享存储器要慢，但是它具有缓存，并且无须考虑冲突问题，主要用来加速对常数的访问。

最后是全局存储器（global memory），使用的是普通的显存，整个网格中的任意线程都能读写全局存储器的任意位置，但读写速度是最慢的（大约需要 400 个时钟周期，而寄存器只需 1 个时钟周期）。

4.2.2 存储器优化思路

存储器的优化对并程序的表现影响最为重要，优化的目的是为了获得最大的读写带宽，在程序的编写中，最大化的使用读写快的存储器，同时减少对低速存储器的读写。

共享存储器除了可以在块内的线程之间进行数据传递和通信外，还可以作为保存中间数据使用，共享存储器的读写速度和寄存器相当，可以在运行过程中将频繁读写的数据写入共享存储器，得到最终结果后再放入全局存储器保存。实践表明，充分利用共享存储器保存中间数据，相对于直接从全局存储器读写，并程序的执行速度会有几倍甚至几十倍的提高。

Kernel 中的变量默认是放在寄存器中，但是寄存器的空间是非常有限的，寄存器空间不足时，变量就会存放到局部存储器（local memory），局部存储器是在全局存储器开辟的一块空间，读写速度比寄存器慢的多。所以在编写 kernel 方法时，使用的变量要尽量少，对于同一个变量尽量重复使用，尽量将要解决的问题划分为更细粒度的子问题。

寄存器和共享存储器的空间都是非常少的，是编写并行程序最大的瓶颈。如何将现有的问题分解成占用空间最小的若干子问题，是决定并行程序执行效率的关键。

4.2.3 共享存储器的优化

存储器优化的主要思路是，将线程块频繁操作的数据从全局存储器写入共享存储器，线程从共享存储器进行操作，操作完成后将结果写入全局存储器。下面以双三次差值为例来说明。

高分辨率图像的大小为 512×512 ，为每一个像素开辟一个线程，网格大小为 16×16 ，线程块大小为 32×32 ，定义如下：

```
dim3 grid(32, 32);  
dim3 threads(32, 32);
```

利用双三次插值将分辨率 256×256 的图像插值放大到 512×512 的并行程序如下：

```
#define BLOCK_SIZE 32  
#define HALF_BLOCK_SIZE 16  
__global__ void it(float* ply, float* phy)ply 为低分辨率图像的 y 值，phy 为高分辨率图像的 y 值  
{  
    // 获得线程的局部和全局索引  
    short bx = blockIdx.x;  
    short by = blockIdx.y;  
    short tx = threadIdx.x;  
    short ty = threadIdx.y;  
    short gx = BLOCK_SIZE * bx + tx;  
    short gy = BLOCK_SIZE * by + ty;  
  
    //分配共享存储器空间  
    __shared__ float lImgY[HALF_BLOCK_SIZE][HALF_BLOCK_SIZE];  
  
    //计算出每个线程要读入的数据，读入到共享存储器。因为要读入的数据大小  $16 \times 16$ ，当前线程的数目是  $32 \times 32$ ，所以让索引都是偶数的线程读入数据。  
    if(((tx % 2) == 0) && ((ty % 2) == 0))  
    {  
        short txd2 = tx / 2;  
        short tyd2 = ty / 2;  
        lImgY[txd2][tyd2] = ply[(HALF_BLOCK_SIZE * bx + txd2) * LY_SIZE + (HALF_BLOCK_SIZE * by + tyd2)];  
    }  
}
```

```

by + tyd2)];
}
// 同步，等待所有线程读入完成
__syncthreads();

//变量
bool inShareMem;
short m, n;
short s_c, s_r, r, c;
float scl_inv;
float x, y, u, v;
float p[4][4];

//计算线程插值所需要的数据，插值所需原数据的位置坐标 s_r、s_c 和 u、v 值
scl_inv = (float)(LY_SIZE - 1) / (HY_SIZE - 1);
x = gx * scl_inv - bx * HALF_BLOCK_SIZE;
y = gy * scl_inv - by * HALF_BLOCK_SIZE;
s_r = (int)x;
s_c = (int)y;
u = x - s_r; //不能换
v = y - s_c;
if(u < 0)
{
    u += 1;
    s_r -= 1;
}
if(v < 0)
{
    v += 1;
    s_c -= 1;
}
//取出以 s_r、s_c 为中心的 4*4 的块
for(m = 0; m < 4; m++)
{
    for(n = 0; n < 4; n++)
    {
        r = (s_r + m - 1);
        c = (s_c + n - 1);
        //是否在共享存储器中，默认为真。
        inShareMem = true;
        //如果越界说明数据不在共享存储器中
        if((r < 0) || (c < 0) || (r >= HALF_BLOCK_SIZE) || (c >= HALF_BLOCK_SIZE))
        {
            inShareMem = false;
        }
        if(inShareMem)
        {
            //如果在共享存储器，直接读取。
            p[m][n] = limgY[r][c];
        }
        else
        {
            //如果不再共享存储器，到全局存储器读取，并防止越界。
            r = r + bx * HALF_BLOCK_SIZE;
            c = c + by * HALF_BLOCK_SIZE;
            if(r < 0)
            {
                r = 0;
            }
            if(r >= LY_SIZE)
            {
                r = LY_SIZE - 1;
            }
            if(c < 0)
            {
                c = 0;
            }
        }
    }
}

```

```

        if(c >= LY_SIZE)
        {
            c = LY_SIZE - 1;
        }
        p[m][n] = ply[r * LY_SIZE + c];
    }
}
//根据得到的数据进行插值，并保存到全局存储器的对应位置
phy[gx * HY_SIZE + gy] = bicubicInterpolate(p, u, v);
}

```

可见，为插值计算，共开辟了 256K 个线程，每个线程执行相同的过程（kernel），程序能够有条不紊的运行并得到正确结果，是因为每个线程根据自己的块内线程索引（tx、ty），网格内的块索引（bx、by），去读写相应位置的数据，完成自己对应的整个计算的一小部分。

4.2.4 kernel 中变量数目的优化

减少 kernel 中的变量，可以让更多的变量放在寄存器中，减少对全局存储器的读写，提高计算速度。尽量缩减程序中的变量数目，找到可以重复使用的变量，或者一个变量用到多个用途；另外也可尝试对程序进行更细粒度的分解。例如下面稀疏信号求解过程中线性方差组的求解中用到的数组 Aa、x_min、tmpvec，在计算结束后，tmpvec 和 Aa 可以用到后面其它的计算过程中。

```

float Aa[64];
float x_min[8];
float tmpvec[8];
for(fi = 0; fi < a_s; fi++)
{
    x_min[fi] = tmpvec[fi]; //vect
}
for(fi = 0; fi < a_s; fi++)
{
    if(Aa[fi * 8 + fi] != 1.0)
    {
        tmp = Aa[fi * 8 + fi];
        Aa[fi * 8 + fi] = 1.0;
        for(fj = fi + 1; fj < a_s; fj++)
        {
            Aa[fi * 8 + fj] /= tmp;
        }
        x_min[fi] /= tmp;
    }
    for(fk = 0; fk < a_s; fk++)
    {
        if(fk != fi)
        {
            tmp = Aa[fk * 8 + fi];
            for(fj = 0; fj < a_s; fj++) //0
            {
                Aa[fk * 8 + fj] -= tmp * Aa[fi * 8 + fj];
            }
            x_min[fk] -= tmp * x_min[fi];
        }
    }
}

```

```
    }
  }
}
```

4.3 减少数据在主机端和设备端之间的传递

数据在主机端（CPU 和内存）和设备端（GPU 和显存）的传送是通过 PCI 总线实现的，其传输速率远小于 GPU 读写显存的速率，比如，NVIDIA GeForce GTX 280 读写显存的带宽为 141GBps，而 PCI 端口传输的速度只有 8GBps。

所以在程序编写中应该尽量减少这种类型的传输，可以考虑采用如下策略：为了避免中间结果在主机端和设备端之间传输，可以将某些加速比并不高甚至不适合并行的部分进行并行，虽然加速比不高，但是减少了频繁的数据传递的时间，总体时间可能会更少；如果最终生成的是图像数据，可以借助 openCL 将显存的数据直接显示出来，避免了数据从设备端到主机端的传递。

OpenGL 是个专业的图形程序接口，是一个功能强大，调用方便的底层图形库。GLUT（OpenGL Utility Toolkit）库的出现更加方便了 OpenGL 编程，它使我们创建完全独立于窗口系统的 OpenGL 程序成为可能，可以使我们能够在不了解 Windows 或者 Microsoft Windows 窗口系统本身的情况下编写 3D 应用程序。

GLUT 提供了用于界面显示、接受键盘输入、接受鼠标输入、处理窗口变换、周期调用等接口，使用起来非常方便。

由于程序生成的高分辨率图像的 RGB 值存放在显存中，可以使用 OpenGL 直接从显存中显示，从而减少数据在显存和内存之间的传输次数。使用 OpenGL 显示 GPU 中的数据主要需要进过以下几个步骤：

1) 初始化 GLUT。

```
{
    glutInit(argc, argv);
    glutInitDisplayMode(GLUT_RGBA | GLUT_DOUBLE);    //像素类型为 RGBA，双缓冲
    glutInitWindowSize(h_w, h_h);                    //设置窗口大小
    glutCreateWindow("Video Reconstruction");         //设置窗口上方的名字
    glutDisplayFunc(SR_display);                      //图像显示方法名为 SR_display
    glutKeyboardFunc(keyboard);                       //从键盘输入处理方法为 keyboard
    glutReshapeFunc(reshape);                         //窗口大小发生变换时，调用 reshape 方法
    glutTimerFunc(40, OnTimer, 1);                   //每个 40ms 调用 OnTimer 方法一次
    glutIdleFunc(idle);                              //空闲时调用 idle 方法
}
```

2) 分配显存设备。

```
{
    cudaGLSetGLDevice (cutGetMaxGflopsDeviceId());
}
```

3) 创建 OpenGL 的 vbo 对象和纹理。

```
{
    GLint g_Bpp = 4;
    GLuint pbo_buffer;
    glGenBuffersARB(1, &pbo_buffer);
    glBindBufferARB(GL_PIXEL_UNPACK_BUFFER_ARB, pbo_buffer);
    glBufferDataARB(GL_PIXEL_UNPACK_BUFFER_ARB, g_Bpp * sizeof(unsigned char) * h_w * h_h,
NULL, GL_STREAM_DRAW_ARB); //设置 pbo_buffer 空间大小
    //创建用于显示的纹理
    glGenTextures(1, &texid);
    glBindTexture(GL_TEXTURE_2D, texid);
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA8, h_w, h_h, 0, GL_RGBA, GL_UNSIGNED_BYTE,
NULL);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
    glBindTexture(GL_TEXTURE_2D, 0);
}
```

4) 将 OpenGL 缓冲对象注册到 cuda，资源注册是潜在高消耗的，因此通常每个资源只注册一次。

```
{
    struct cudaGraphicsResource *cuda_pbo_resource;
    cudaGraphicsGLRegisterBuffer(&cuda_pbo_resource, pbo_buffer, cudaGraphicsMapFlagsWriteDiscard);
}
```

5) 一旦资源被注册到 CUDA，就可以按需要被任意次的映射和解映射，映射和解映射使用 `cudaGraphicsMapResources()` 和 `cudaGraphicsUnmapResources()`。被映射的资源可以通过方法 `cudaGraphicsResourceGetMappedPointer` 得到指向设备的指针 `rgba`，`kernel` 通过 `rgba` 修改映射资源中的数据。

`void SR_display()`

```
{
    unsigned int *rgba;
    cudaGraphicsMapResources(1, &cuda_pbo_resource, 0);
    size_t num_bytes;
    //映射数据，将 pbo 资源的地址映射到 rgba
    cudaGraphicsResourceGetMappedPointer((void **)&rgba, &num_bytes, cuda_pbo_resource);
    // ycrb2rgb 为 kernel 方法，将最终数据存放到 rgba 指向的空间
    ycrb2rgb<<<grid, threads>>>(psry_d, phcr_d, phcb_d, cntMat_d, rgba);
    //解除映射
    cudaGraphicsUnmapResources(1, &cuda_pbo_resource, 0);

    glClear(GL_COLOR_BUFFER_BIT);
    //从 pbo 加载纹理
    glBindBufferARB(GL_PIXEL_UNPACK_BUFFER_ARB, pbo_buffer);
    glBindTexture(GL_TEXTURE_2D, texid);
    glTexSubImage2D(GL_TEXTURE_2D, 0, 0, 0, h_w, h_h, GL_RGBA, GL_UNSIGNED_BYTE, 0);
    glBindBufferARB(GL_PIXEL_UNPACK_BUFFER_ARB, 0);
    glBindProgramARB(GL_FRAGMENT_PROGRAM_ARB, shader);
    glEnable(GL_FRAGMENT_PROGRAM_ARB);
    glDisable(GL_DEPTH_TEST);

    //设置观测角度
    glBegin(GL_QUADS);
    glTexCoord2f(0, 0);
    glVertex2f(0, 1);
    glTexCoord2f(1, 0);
    glVertex2f(1, 1);
    glTexCoord2f(1, 1);
    glVertex2f(1, 0);
    glTexCoord2f(0, 1);
    glVertex2f(0, 0);
    glEnd();
    glBindTexture(GL_TEXTURE_2D, 0);
    glDisable(GL_FRAGMENT_PROGRAM_ARB);
    //更新缓冲
    glutSwapBuffers();
}
```

6) 注销 OpenGL 缓冲对象。

```
{
    cudaGraphicsUnregisterResource(cuda_pbo_resource);
    glDeleteBuffersARB(1, &pbo_buffer);
    glDeleteTextures(1, &texid);
}
```

4.4 配置优化与指令优化

配置优化就是设计如何让 GPU 上的所有处理器都处于忙碌的状态。比如因为每个流多处理器对应的共享存储器的大小是固定的，编译器会根据每个块内线程需要的共享存储器的大小计算可以同时执行的块数；用户定义的网格的维数和块的维数决定同时可以执行的线程数。

任何分支指令（如 if、switch、do、for、while）会使同一个束（warp）中的线程产生分支，即不同线程可能执行不同的路径，在这种情况下，不同的分支路径会被依次执行，直到分支路径执行完后再汇总到同一个路径，这样会增加所有线程的指令执行数目。

4.5 各个阶段的加速比及分析

为了得到的数据更精确，选择了三张图片进行时间统计。各个阶段的串、并程序耗时和加速比如下表：

	双三次插值			信号重建			总体		
	串	并	加速比	串	并	加速比	串	并	加速比
第一次	281	6.7726	41.5	2875	77.08	37.3	3170	90	35.2
第二次	297	6.7957	43.7	2876	79.14	36.34	3203	90	35.5
第三次	297	6.9	43.04	2875	78.3	36.72	3169	94	33.71

可以看到，双三次插值获得了比信号重建更好的加速比。信号重建算法较双三次插值的算法复杂，kernel 方法中使用的变量更多，无法全部放入寄存器，部分变量会存放在局部存储器中，影响数据的读写速度。可见将程序进行更细粒度的分解往往能获得更好的效果。

通过设计并行程序，将每幅图像的重建时间从 3200 毫秒减少到 90 毫秒，

重建视频可以达到 10 帧每秒的帧率,但是目前重建思路依然是针对单幅图像重建,还没有考虑视频本身的特性,下一章将从视频的解码出发,找到帧间的冗余数据,减少重建过程的计算量,进一步提高帧率。

第五章、压缩视频编解码及其在视频超分辨率重建中的应用

实际应用中最常见的视频格式都是压缩格式，如 MPEG 系列、H. 26x 等压缩标准。

5.1 视频编码格式与视频文件格式

所谓视频编码格式就是指通过特定的压缩技术，将某个视频格式的文件转换成另一种视频格式文件的方式。比较常见的有 H. 26x 系列和 MPEG 系列。

H. 26x 和 MPEG-X 都是应用最广泛的视音频压缩国际标准[32]。其中，H. 26x 是活动图像编码标准，而 MPEG 则主要包括 MPEG 视频、MPEG 音频和 MPEG 系统(视音频同步)3 个部分的标准。H. 26x 和 MPEG-X 既有区别又有紧密的联系，特别是 H. 262 和 H. 264，它们既是 ITU-T 制定的标准，又分别是 ITU-T 与 ISO/IEC 联合制定的 MPEG-2 和 MPEG-4 的组成部分。

MPEG-4 和 H. 264 对于视频压缩的思路是相同的，只是针对应用的不同背景，对不同的内容有所侧重。MPEG-4 视频部分强调灵活性，而 H. 264 则强调有效性和可靠性。MPEG-4 视频编码集成了合成的、自然的、视频的、图像的 2D 和 3D 图形对象，展现出的丰富的、交互的、在线的世界，H. 264 则可以提供高效、可靠的视频通信，支持双向的、流式的和广播的应用，同时对信道传输差错具有鲁棒性。

视频是现在多媒体系统中的重要组成部分。为了适应储存视频的需要，设定了不同的视频文件格式来把视频和音频放在一个文件中，以方便同时播放。

视频文件格式有不同的分类，如：

微软视频：wmv、asf、asx

Real Player：rm、rmvb

MPEG 视频：mpg、mpeg、mpe

手机视频：3gp

Apple 视频：mov

Sony 视频：mp4、m4v

其他常见视频：avi、dat、mkv、flv、vob。

视频编码格式与视频文件格式没有对应的关系,比如 avi 格式的视频文件,其编码格式既可以是 DIVX 格式、XVID 格式,也可以是 ffdshow MPEG-4 等格式。视频文件相当于一个容器,里面可以包裹不同的视频和音频编码格式。

Xvid 是一个开放源代码的 MPEG-4 视频编解码器,它是基于 OpenDivX 而编写的。是目前最常用的视频编码解码器(codec),而且是第一个真正开放源代码的,通过 GPL 协议发布。XviD 的文件扩展名可以是 AVI、MKV、MP4 等。

由于 Xvid 的开放性,所以有了从源码出发视频编解码的原理的可能。Xvid 的开源代码可以在 Xvid 的官网下载 (<http://www.xvid.org/>),此外还有一些比较好的参考资料[33][34][35]。

5.2 MPEG-4 视频编解码思想

视频编码的主要目的是找到并消除视频中的冗余,主要经过三个步骤:

- 1、消除帧间冗余。由于视频的帧率比较高,相邻帧之间大部分甚至全部内容都是相同的;
- 2、消除帧内冗余。在视频的某一帧内,相邻样本点之间的像素值往往是相同活相近的,即具有很强的相关性,所以未处理的自然图像是很难压缩的。
- 3、变换编码与量化、熵编码。

5.2.1 宏块的概念

宏块是视频编解码的基本单位,宏块(MacroBlock)是帧中大小为 16×16 的像素区域,包含 256 个亮度样本(4 个 8×8 的块),64 个蓝色度样本(1 个 8×8 的块)和 64 个红色度样本(1 个 8×8 的块),一共 6 个 8×8 的块。

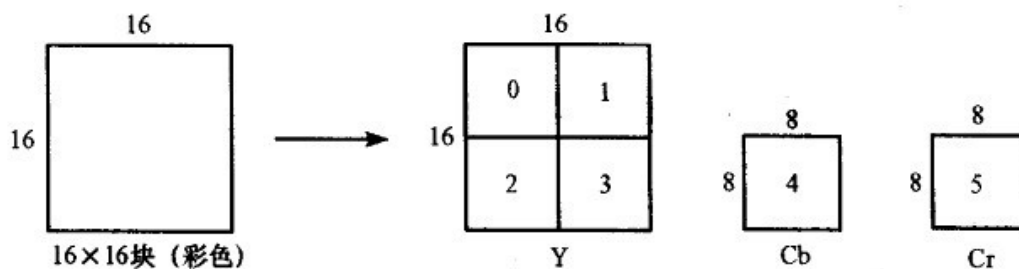


图 5-1 宏块的组成

在 Xvid 源码中的结构体定义如下

```
typedef struct
{
    VECTOR mvs;
    short int pred_values[6];
    int mode;
    VECTOR pmvs;
    int32_t sad8[4];
    int32_t sad16;
    int cbp;
}MACROBLOCK;
```

其中：

mvs 表示当前宏块的运动矢量。

pred_values 表示 6 个 8×8 小块的 DC 系数值，作为后续 I 块的预测值。

mode 表示当前宏块的编码模式，MODE_INTER 和 MODE_INTRA 和 MODE_NOT_CODED 这三中模式之一。

pmvs 表示当前宏块运动矢量减去预测值后的差值。

sd8[4] 表示 16×16 宏块中 4 个 8×8 小块的 SAD 值。

sad16 表示 16×16 宏块的 sad 值。

cbp 表示当前宏块的 cbp 值。

5.2.2 帧间冗余的消除

在时域模型中，当前帧减去预测值得到残差图像，如果预测帧越精确，所得到的残差图像能量也就越小，压缩后的体积也就越小。残差图像经过编码后被传输到解码器，解码器通过参考帧与残差图像相加来恢复当前帧图像。

帧间冗余的消除是通过运动估计和运动补偿来实现的，具体步骤如下：

- 1、在参考帧（一般是当前帧的前一帧）中搜索 $M \times N$ 个宏块，将当前宏块与搜索区域的部分或所有宏块进行比较，找到最佳匹配块。寻找最佳匹配块常用的思想是计算当前块与比较块相减后的残差能量块，使得残差块能量最低的比较块即为最佳匹配块，搜索最佳匹配块的过程称为运动估计

- 2、将最佳匹配块作为当前块的预测块，当前块减去预测块后得到相应的 $M \times N$ 个残差块（运动补偿）。

- 3、对残差块进行编码和传输，同时预测块和当前块的相对位置也被传输给

解码器。

解码器根据运动矢量寻找预测块，与解码后的残差块相加，以重建原始图像块。

以宏块为基本单位，运动估计过程是在参考帧中寻找与当前宏块最匹配的宏块，以参考帧的当前宏块位置为中心寻找匹配规则达到最小值的最佳匹配，如图 5-2。

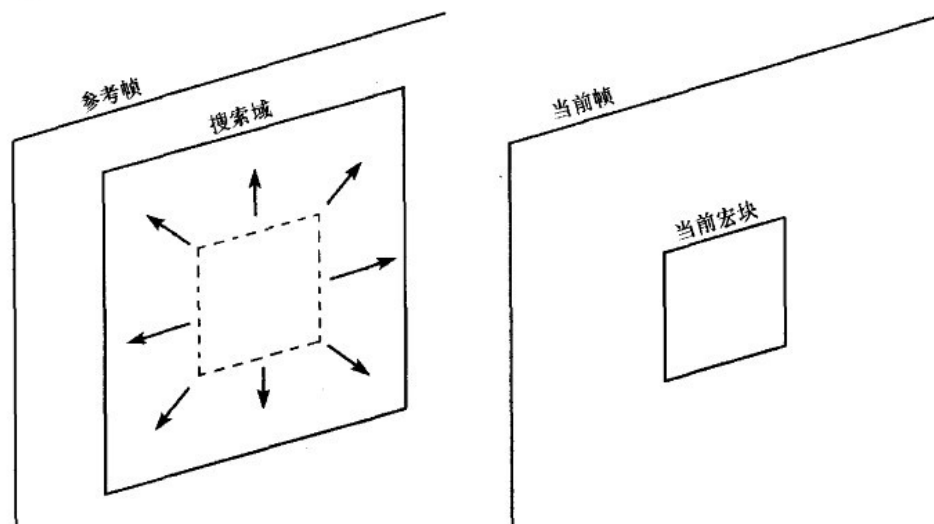


图 5-2 在参考帧中搜索最佳匹配块

5.2.3 帧内冗余的消除

图像样本区域可以用同一帧图像中已传输的样本（宏块）来预测，又称预测编码。如下图所示，如果当前帧以光栅次序编码，则无论在编码端还是解码端，宏块 A、B 和 C 都在 X 之前被编码和解码。

根据这些值，编码器按照某种公式产生 X 的预测，X 的实际值与预测值相减得到相应的残差（X 与预测值相减的结果），并对残差进行编码，由于残差能量相对较少，从而达到压缩的目的。在解码端，根据已解码的宏块 A、B 和 C 的值计算 X 的预测值，与解码后的宏块 X 的残差相加得到 X 的重建值。

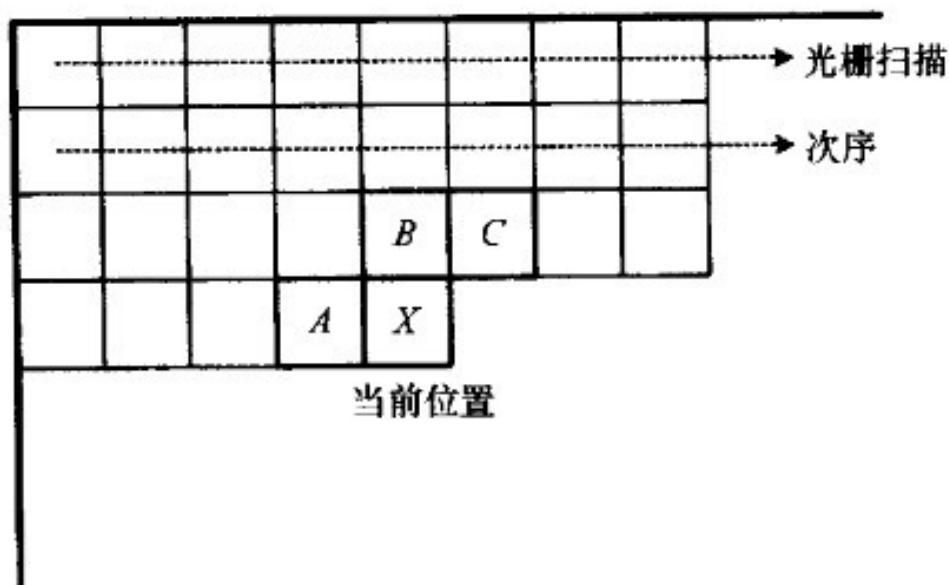


图 5-3 帧内预测编码

5.2.4 变换编码与量化、熵编码

变换编码是将图像或运动补偿后的残差图像数据转换到另外一个域上，变换域的数据必须是不相关的、紧凑的，主要能量集中在少数的几个分量上，实现压缩的目的。最长见的是离散余弦变换和小波变换，对变换后的参数进行量化和游程编码，可得到代表图像绝大部分信息的参数。

熵编码是将描述视频流的一串符号编码成适于传输或存储的压缩比特流，熵编码的输入有量化后的系数、运动矢量（运动补偿在 x 方向和 y 方向上的偏移）、头信息（宏块头信息，帧的头信息，流的头信息等）。熵编码利用信息的相关性和出现频率的大小进行进一步的压缩，如变长编码和预测编码等。具体可以参考文献[33]。

5.3 寻找帧间冗余信息

视频编码中帧的类型主要分为 I 帧（关键帧）和 P 帧（参考帧），I 帧是依靠帧内冗余实现压缩，通过本身已解码的宏块对未解码的宏块进行预测；P 帧是参考前一帧，通过运动估计和运动补偿得到，通过编码运动补偿的值（就是和前一帧最相似的宏块之差）进行压缩。

在 5.2.1 节宏块的结构体定义中的 mode 变量表示该宏块的类型,常见的有三种类型: MODE_INTRA 、 MODE_INTER 和 MODE_NOT_CODED。其中, MODE_INTRA 表示该宏块是帧内编码实现; MODE_INTER 表示通过帧间编码实现; MODE_NOT_CODED 表示该宏块是运动估计和运动补偿均为 0 的帧间编码宏块。

I 帧中的宏块均为 MODE_INTRA 类型,没有帧间冗余,P 帧中则包括 MODE_INTER 和 MODE_INTRA 和 MODE_NOT_CODED 三中模式的宏块。其中类型为 MODE_NOT_CODED 的宏块,是运动估计和运动补偿都为 0 的宏块,就是说和前一帧同一位置的宏块完全相同,这个就是要找的帧间冗余,在重建帧的时候对该块不进行重建,从而减少计算量。

具体实现是在 Xvid 源码中得到宏块类型分布矩阵,比如,将编码模式为 MODE_INTER 和 MODE_INTRA 标记为 0,模式为 MODE_NOT_CODED 的标记为 1。将该矩阵传入到 GPU 的地址空间中,在图像重建内核中读入该矩阵,对于类型为 MODE_NOT_CODED 的块,直接保留前一帧对应宏块的值,而当前帧的对应宏块不进行重建,从而减少计算量。

下面展示了通过该思想得到的帧间冗余,如图 5-4 和 5-5,其中上面 5 幅图像是原始帧,下面的 5 幅连续图像是标注了冗余的帧:



图 5-4 帧间冗余相对较少的视频序列

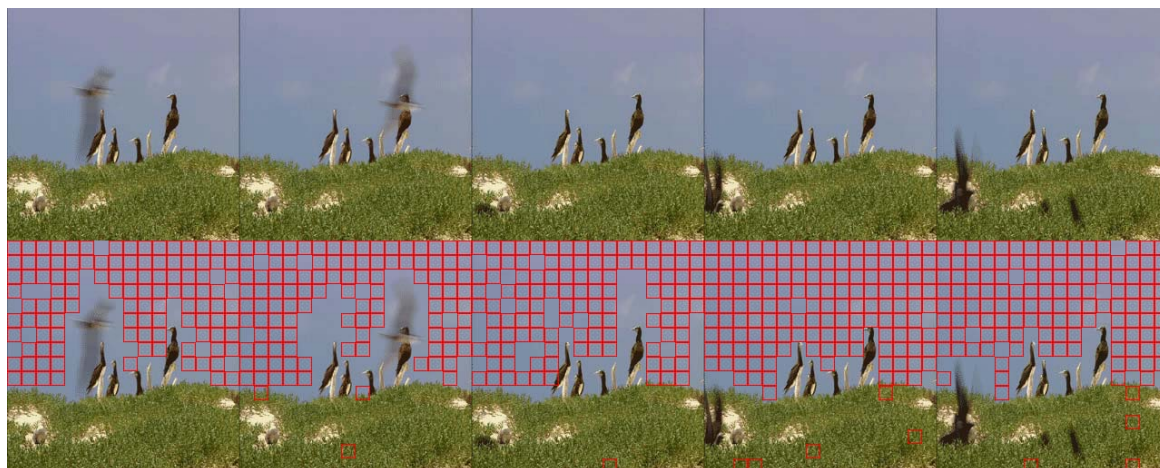


图 5-5 帧间冗余相对较多的视频序列

图中用红色方框标记出来的就是当前帧相对于前一帧的帧间冗余，即该宏块和前一帧相同位置的宏块内容是相同的。所以在对宏块进行重建的时候，只需保留上一帧在相同位置的值，无需重复重建。

实验结果表明，视频帧率的大小取决于帧间冗余的多少：图 5-4 示例是帧间冗余较少的帧序列，对于该类型的帧序列可以达到 12 帧每秒的帧率；图 5-5 示例是帧间冗余较多的帧序列，可以达到 20 帧每秒的帧率。

第六章、总结与展望

6.1 重建效果对比

在第三章的图 3-2 中对基于压缩感知的重建算法和传统重建算法的重建效果进行了对比。本节主要对重建后的视频帧和播放器拉伸后的视频帧效果进行对比。

图 6-1 列出的四帧是帧间冗余相对较少的帧，对于此类帧的重建可以达到 12 帧每秒的重建帧率。通过对比可以发现，基于压缩感知思想重建后的帧比视频播放器播放的效果更清晰。





图 6-1 重建效果与播放器效果对比（共列出四组图像，左边的为重建后的效果，右边的为 Windows Media Player 播放器效果）。

图 6-2 列出的四帧是帧间冗余相对较多的帧，可以看到背景中的蓝天在相邻帧之间是没有变化的，帧间的冗余可以达到整个帧数据中的一半，对于冗余的数据，直接保留前一帧的重建结果，而当前帧不需进行重建，大大减少了计算量，通过时间统计发现，对于此类帧的重建可以达到 20 帧每秒的帧率。

通过对比山上的植被可以发现，重建后图像中的植被更加清晰，展现的细节部分更多。



图 6-2 算法重建效果与播放器效果对比

动物皮毛往往能展现更多的细节信息，对比效果如下：



图 6-3 重建效果与播放器效果对比

6.2 本文工作总结

超分辨率重建的应用背景是比较广泛的，本文的图像重建是针对图像放大过程的重建：对一个低分辨率视频，在拉伸放大后仍然能保持较多的细节信息。目前主流的视频重建思路有：针对视频压缩过程引起图像质量退化因素的重建；传统的重建思想，即对解码后的帧进行基于图像的超分辨率重建。

压缩理论是 2006 年提出的一个比较热门的理论，被广泛的应用在图形学的各个领域。本文将压缩感知理论应用到图像的超分辨率重建上，结果表明：基于压缩感知理论的图像重建相对于传统的图像重建算法获得了更好的效果，展现了更详细的细节信息。

压缩感知理论的特性决定了重建过程是相对复杂的，所以本文采用了 GPU 加速的思想，用并行计算的思路完成图像重建，通过对并行程程序的编写与优化，获得了较高的加速比，为将算法应用到视频重建提供了条件。

数据在主机端和设备端之间的传送是通过 PCI 接口实现，该接口的带宽相对较低，为了减少数据在两者之间的传送，借助 OpenGL 和 CUDA 的交互，将 GPU 计算生成的结果数据直接在显卡上显示出来，减少了数据在主机端和设备端之间的传输，增加了重建帧率。

目前的主流视频都是压缩视频，压缩的目的是减少视频的帧间冗余和帧内

冗余。在视频的编解码过程中,这种冗余信息会有所体现。MPEG-4 是目前主流的视频编码格式,Xvid 是一种开源 MPEG-4 视频编解码标准,通过分析 Xvid 源码,找到了可以利用的冗余信息,即当前帧相对于前一帧没有任何变化的部分,对于冗余信息,只需保留参考帧的数据,在当前帧无需重建,减少了重建过程的计算量。

通过以上种种优化策略,实现了对于分辨率为 256×256 的低分辨率的实时重建(平均 15 帧每秒),相对于一般播放器(如 Windows Media Player)拉伸后的效果,图像质量有了明显的提高(见 6.1 节效果对比)。

6.3 未来工作展望

基于压缩感知的视频重建为“高清视频”的实现提供了一个思路。但是在具体应用还是很多限制条件,这是由算法本身的特点和硬件环境局限性决定的:

1)从图 3-1 的算法流程可知,压缩感知理论中的稀疏矩阵是通过机器学习的方式得到的,本文选择的训练图片针对的是自然场景的图片,就是说对于一般自然场景的图片,该矩阵是稀疏的。但是针对其它内容的图像,该矩阵未必是稀疏的,得到的重建图像质量不能保证。

2)从 2.2 节压缩感知的数学模型可以发现,信号的重建过程是相对复杂的。本文重建的低分辨率图像大小为 256×256 ,可以实现 15 帧每秒的帧率。当原始图像的分辨率增加后,计算量会相应的增加,在现有的硬件环境下实现实时重建会比较困难。

3)并程序的网格大小和块的大小,在程序运行之前就是固定的,运行过程中不能动态修改,所以图像放大的倍数是固定的。

4)从 5.3 节压缩视频帧间冗余的需找过程可以发现,每一帧的帧间冗余是不固定的,重建过程的计算量的多少取决于当前帧间冗余的多少,对帧间冗余相对较多的帧,重建较快,反之,帧率较慢,所以在重建过程中帧率是不固定的。

参考文献

- [1] Patti A J, Altunbasak Y. "Super-resolution image estimation for transform coded video with application to MPEG". Proceedings of IEEE International Conference on Image Processing, 1999.
- [2] Ahunbasak Y, Patti A J. "A maximum a posteriori estimator for high resolution video reconstruction from MPEG video". Proceedings of the International Conference on Information Processing[C], 2000.
- [3] Segall C A, Molina R, Katsaggelos A K. "High-resolution images from low-resolution compressed video". IEEE Signal Processing Magazine, 2003, 20(3): 37-48.
- [4] Merino M T, Nunez J. "Super-resolution of remotely sensed images with variable-pixel linear reconstruction". IEEE Transactions on Geoscience and Remote Sensing, 2007, 45(5): 1446-1457.
- [5] Harris J.L., "Diffraction and Resolving Power". J.O.S.A, 1964, 54(7): 931-936.
- [6] Goodman J.W., "Introduction to Fourier Optics". McGraw_Hill, New York, 1968.
- [7] R. Y. Tsai and T. S. Huang, "Multiframe image restoration and registration". Advances in Computer Vision and Image Processing, Greenwich, JAI Press, 1984, 1(7): 317-339.
- [8] S. P. Kim and W. Y Su, "Recursive High-resolution Reconstruction of Blurred Multi-frame Images". IEEE Trans. Image Processing; Oct.1993,2(2).
- [9] Irani, M.and Peleg,S. , "Motion Analysis for Image Enhancement: Resolution, Occlusion, and Transparency". Journal of Visual Communication and ImageRepresentation, 2004, 4(4):324-335.
- [10] A.J. Patti, M.I. Sezan, A.M. Tekalp, "Robust Methods for High-quality Stills from Interlaced Video in the Presence of Dominant Motion". IEEE Trans Circuitsand Systems for Video Technology, 1997,7(2):328.342.
- [11] S Borman, R L Stevenson. "Simultaneous Multi-frame MAP Super-resolution

Video Enhancement Using Spatio-temporal Priors". Image Processing, 1999. ICIP 99. Proceedings. 6(2):469•473.

[12] J. Yang, J. Wright, T. Huang, and Y. Ma, "Image super-resolution as sparse representation of raw image patches". in Proc. IEEE Conf. Comput. Vis. Pattern Recognition., 2008, pp. 1–8.

[13] Jianchao Yang, John Wright, Thomas S. Huang, and Yi Ma, "Image Super-Resolution Via Sparse Representation". IEEE Transactions On Image Processing, Vol. 19, No. 11, November 2010.

[14] M. Andreut. "Fast GPU Implementation of Sparse Signal Recovery from Random Projections". Institute for Biocomplexity and Informatics University of Calgary.

[15] Borghi, A., Darbon, J., Peyronnet, S., Chan, T.F., Osher, S., "A simple compressive sensing algorithm for parallel many-core architectures". CAM Report 08-64, UCLA(2008)

[16] E Candès, Compressive sampling. "Proceedings of the International Congress of Mathematicians". Madrid , Spain , 2006, 3 :1433 - 1452.

[17] E Candès, J Romberg, Terence Tao. "Robust uncertainty principles: Exact signal reconstruction from highly incomplete frequency information". IEEE Transaction Information Theory, 2006, 52 (2):4892509.

[18] E J Candès and T Tao. "Near optimal signal recovery from random Projections: Universal Encoding Strategies? ". IEEE Transaction Information Theory, VOL. 52, NO. 12, DECEMBER 2006.

[19] Donoho, D. L., "Compressed Sensing". IEEE Transactions on Information Theory, V. 52(4), 1289–1306, 2006.

[20] 喻玲娟, 谢晓春, “压缩感知理论简介”。电视技术。2008 年, 第 12 期。

[21] 李树涛, 魏丹, “压缩传感综述”。自动化学报。2009 年 11 月。

[22] John Wright, Allen Y. Yang, Arvind Ganesh, ect. "Robust Face Recognition via Sparse Representation". IEEE Transactions on Pattern Analysis and Machine Intelligence. 2008.

[23] Jianchao Yang, Kai Yu, etc. "Linear Spatial Pyramid Matching Using Sparse

Coding for Image Classification". Computer Vision and Pattern Recognition, 2009. CVPR 2009.

[24] Donoho D L. "For most large underdetermined systems of linear equations: the minimal l_1 -norm solution is also the sparsest solution". Communications on Pure and Applied Mathematics, 2006,59(6):797-829.

[25] Kim S, Koh K, Lustig M, Boyd S, Gorinevsky D. "An interiorpoint method for large-scale L_1 regularized least squares". IEEE Journal of Selected Topics in Signal Processing, 2007,1(4): 606-617.

[26] Figueiredo M A T, Nowak R D, Wright S J. "Gradient projection for sparse reconstruction: Application to compressed sensing and other inverse problems". IEEE Journal of Selected Topics in Signal Processing, 2007, 1(4): 586-598.

[27] Donoho D L, Tsaig Y. "Fast solution of l_1 -norm minimization problems when the solution may be sparse", Technical Report, Department of Statistics, Stanford University, USA,2008.

[28] Tropp J, Gilbert A. "Signal recovery from random measurements via orthogonal matching pursuit". Transactions on Information Theory, 2007, 53(12): 4655-4666.

[29] Bradley Efron, Trevor Hastie, Iain Johnstone and Robert Tibshirani, "Least Angle Regression". The Annals of Statistics 2004, Vol. 32, No. 2, 407.

[30] Honglak Lee, Alexis Battle, Rajat Raina, Andrew Y. Ng. "Efficient sparse coding algorithms". Computer Science Department Stanford University.

[31] NVIDIA Corporation, "CUDA C Programming Guide", "CUDA C Best Practices Guide".

[32]裴智勇, 张春红。 “H.26x 与 MPEG-x”。电信工程技术与标准化, 2005年2月。

[33] I.E.G. Richardson. "H.264 and MPEG-4 Video Compression: Video Coding for Next-generation". Wiley (2003).

[34]毕厚杰。“新一代视频压缩编码标准-H.264/AVC”。北京: 人民邮电出版社, 2005。

[35]杨书良。“Xvid 编码器源代码剖析”。

[36]石光明，刘丹华，高大化。压缩感知理论及其研究进展。电子学报，2009年5月。

致谢

首先感谢我的指导老师向辉老师。毕业在即，回顾三年多来向老师在学习、科研、生活上对我的关心和帮助，深为感激。学习上，向老师要求严格、悉心教导，让我们接触更多的理论知识，为科研做了很好的铺垫；科研方面，向老师思路开阔、意识创新，没有给我们过多的思想包袱，引导我们深入研究，取得了一定的成果；生活方面，向老师时刻关心，同时跟我们探讨做人做事的道理，使我受益良多。

感谢山东大学人机交互与虚拟现实中心的汪嘉业老师、孟祥旭老师、李学庆老师、龚斌老师、屠长河老师、杨承磊老师、潘荣江老师、秦学英老师、彭京亮老师、刘士军老师、蒋志芳老师、徐延宁老师、武蕾老师、王璐老师等。老师们为实验室提供了浓厚的科研学术氛围、自由的实验环境；学术讨论班上与我们共同探讨科研问题，并给出相应的指导和建议；同时通过老师们的学术关系引进各学术领域的专家给我们开展讲座，让我们接触到了学术前沿技术，受益匪浅。

感谢我的辅导员贺平老师，学生工作处刘宏老师、李雪梅老师等。三年里各位老师为我们学习、科研、生活提供了很多便利和帮助。

感谢研究生期间的同学和朋友们。感谢戴燕飞师姐、刘梅师姐、荣忠旭师兄、林蒋师兄、王延可师兄、陆鹏师兄、冯文娟师姐、张文秀师姐等在科研上对我的指导；感谢李想、吴国圣、张正海、朱燕、李乔宇等实验室的同学在共同的科研、学习中提供的帮助；感谢刘吉庆、石义波、于德海等师弟师妹在学习、生活上的轻松相处、相互关心。

最后，感谢家人对我的支持和鼓励。感谢他们这么多年来对我学业上的鼓励和生活上的支持。他们为我付出了很多很多，是我的坚固后盾和精神支柱。

攻读学位期间发表的学术论文目录

- [1] "A GPU Accelerated Algorithm for Compressive Sensing Based Image Super-Resolution". Digital Media and Digital Content Management, Workshop on, pp. 198-202, 2011 Workshop on Digital Media and Digital Content Management, 2011, 1st author, Published.
- [2] "A GPU Accelerated Algorithm for Compressive sensing based Video Super-resolution". International Conference on Audio, Language and Image Processing, 2012, 1st author, Contributed.

攻读学位期间参加的项目

- [1] 基于通用计算技术的图像标注与检索关键技术研究，2010.5-2011.5，AMD
公司合作项目项目。
- [2] 医学虚拟现实关键技术研究，No. 2011JC020，山东大学自主创新基金资助。

学位论文评阅及答辩情况表

学位论文评阅人	姓 名		专业技术 职 务	是否博导 (硕导)	所 在 单 位		总体评价※
答辩委员会成员	姓 名		专业技术 职 务	是否博导 (硕导)	所 在 单 位		
	主席						
	委 员						
答辩委员会对论文的 总体评价※			答辩秘书		答辩日期		
备注							

※ 优秀为“A”；良好为“B”；合格为“C”；不合格为“D”。